Voor jouw apotheek

Svelte Fundamentals

Module – State Management

Peter Kassenaar – info@kassenaar.com

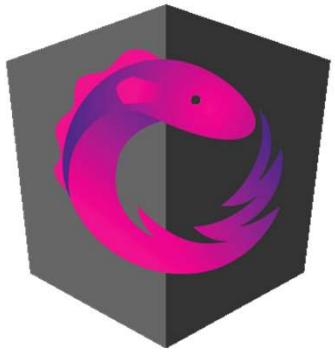
eduvision
opleiding & training

State Management

Sharing data (e.g. ‘state’) over multiple components

What is State Management?

- Various design patterns, used for managing *state* (data in its broadest sense!) in your application.
- Multiple solutions possible – depends on application & framework



Examples of ‘state’

- State is simply
  - *‘data that determines what the UI shows’*
- Examples:
  - User logged in/logged out
  - Contents of a shopping cart
  - Selected theme, dark/light, other configuration stuff
  - API data, shared across components to prevent multiple http-calls

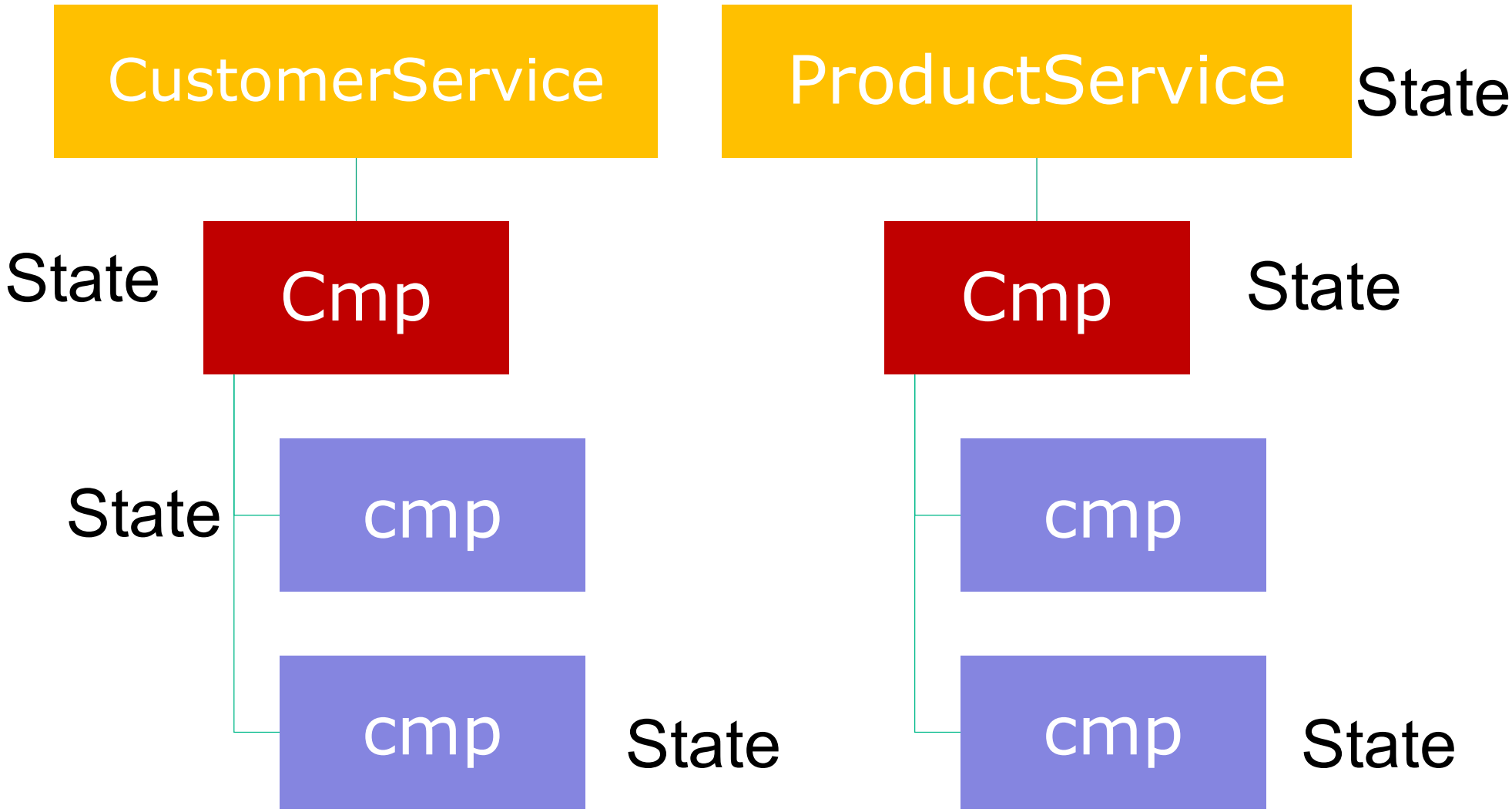
Two types of State

- Local state:
  - Belongs to one component
  - Stored inside that component
  - Only the component reacts to changes
  - Example: `let count = $state(0)`
- Shared/global state
  - Used by multiple components
  - Stored outside components in a Store (typically `xxxStore.ts`)
  - Example: `export const count = writable(0)`

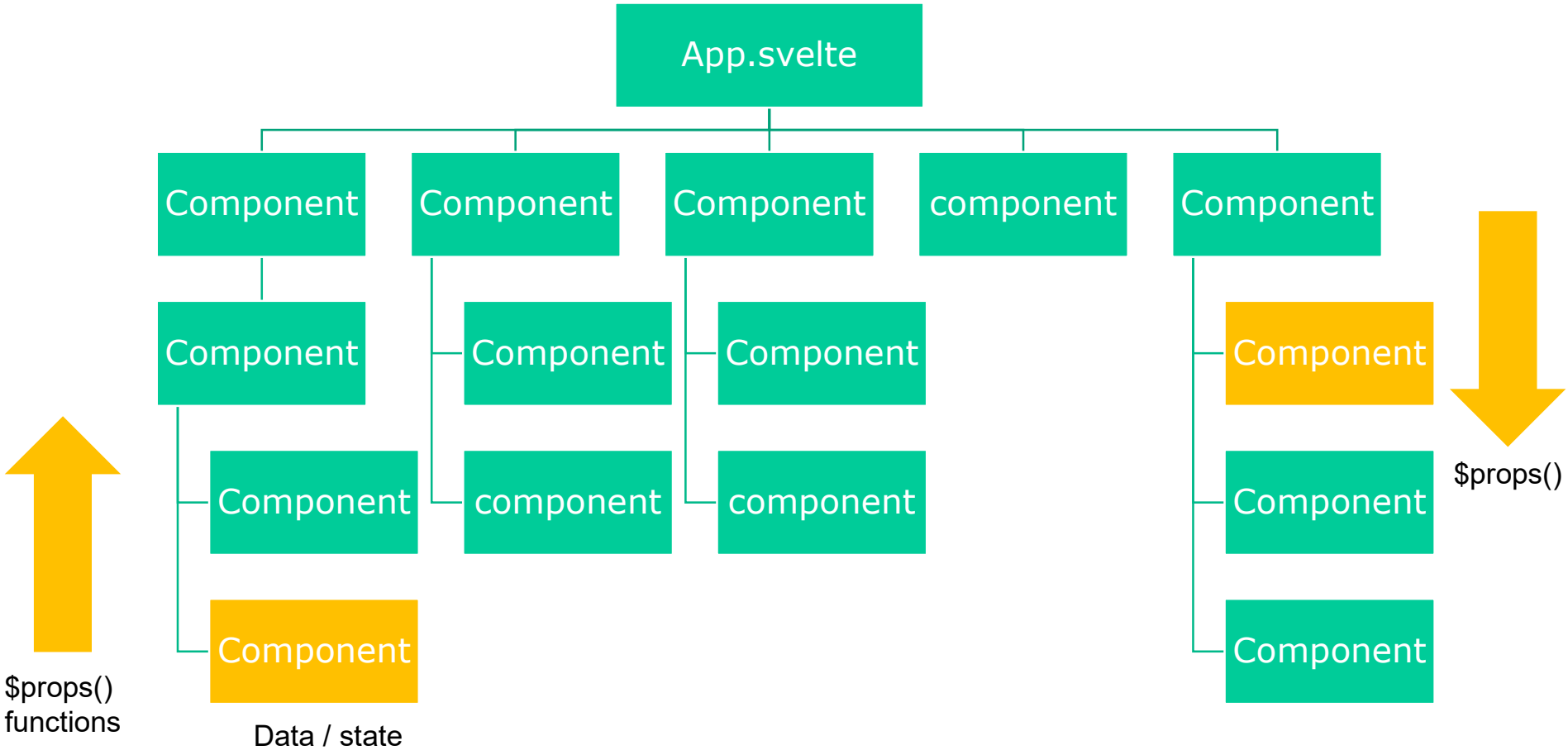
State: Single Source of truth

- By using a state mechanism, your application has a *single source of truth*
  - All component read and write to the same state variables
- Shared reactive state:
  - `writable<T>` → read and write/update
  - `readable<T>` → read-only
  - `derived<T>` → computed from other stores/values
- `$state()` automatically subscribes
  - State is automatically updated everywhere

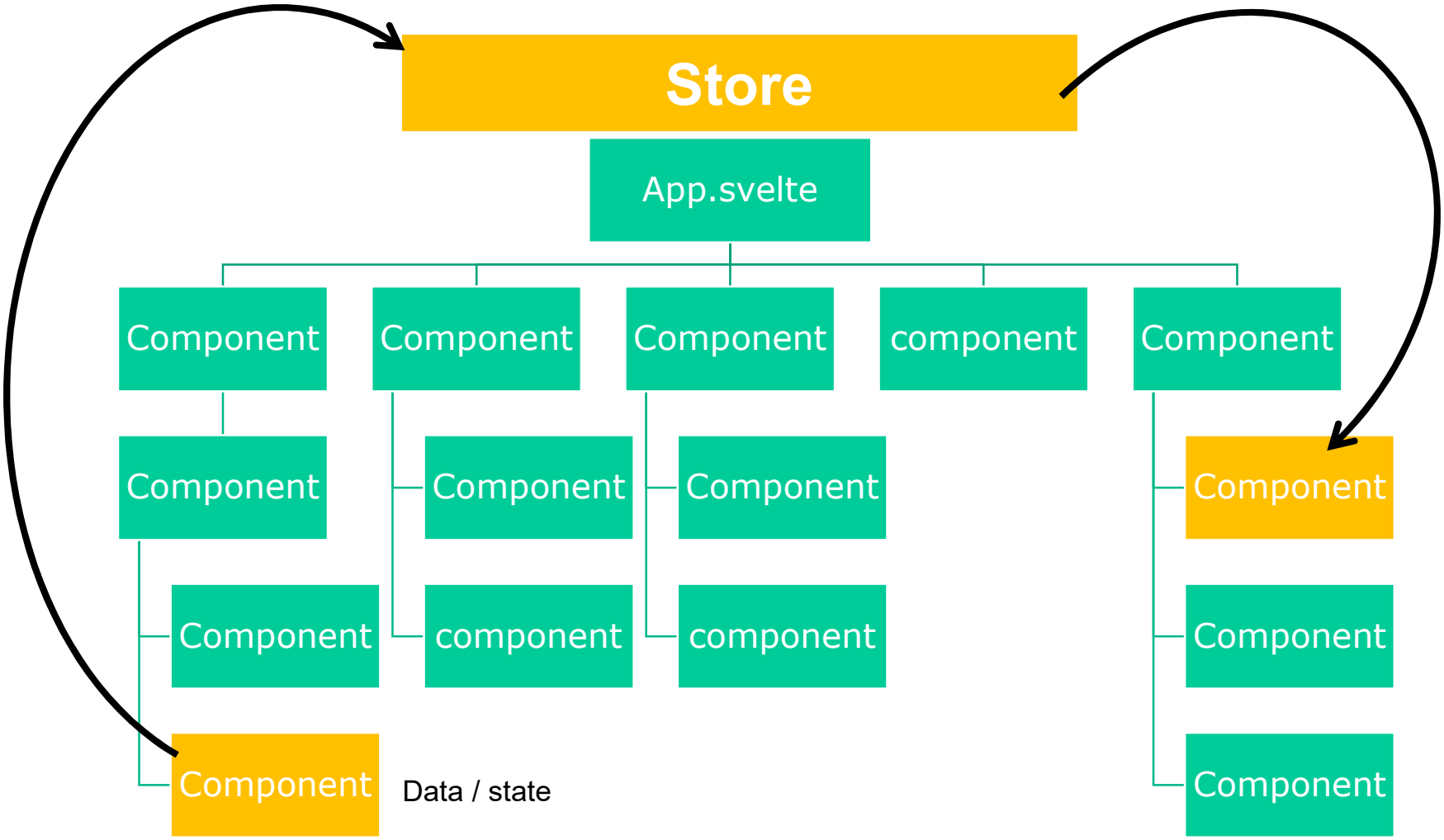
State management without a store



Data flow in complex applications



State management *with* a store



Benefits of using a store

- State is only changed in a controlled way
- Component state is also driven from the store
- Based on immutable objects – b/c they are predictable
- In Svelte – state management is much easier than in React / Angular
  - No Redux-like systems
  - You CAN if you want to (it is all JavaScript), but no one does that
  - No reducers, actions, `dispatch()`, etc.

Svelte 5 stores - mental model

- *no* global store provider
- *no* reducers
- *no* context wrappers
- *no* selectors
- Just: shared store module + `$state()`
  - (in modern runes store)

Two types of stores

1. Classic Store

2. Svelte 5 store using runes

Classic Store

Exporting a store file with just properties that you can import – easy and used a lot

Simple example – using a `counterStore`

- 3 steps:
  - 1. Create store
  - 2. Read store
  - 3. Update store
- This is now also called a 'classic store'
  - Svelte 5 – using top-level `$state()` runes.

```
// 1. counterStore.ts
import {writable} from "svelte/store";

// the 'writable' function creates a store/reactive state
container that can be written to
export const counter = writable(0);
```

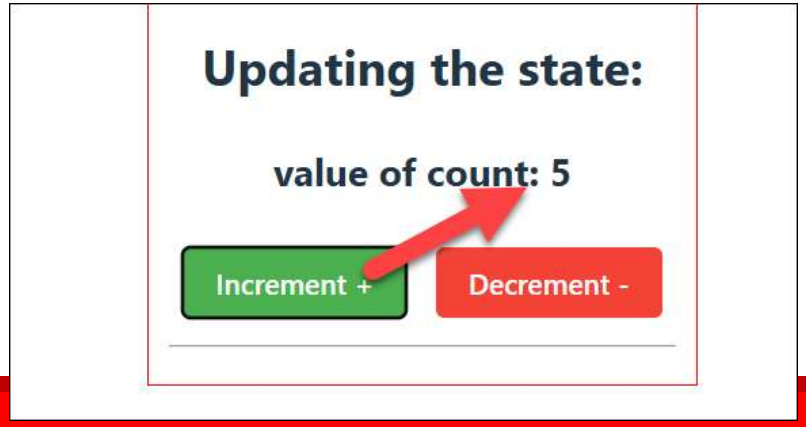
Updating the store – import + create functions

```
<script lang="ts">
    // importing the counter from a global store
    import {counter} from "./store/counterStore";

    // functions to update the counter.
    const increment = () => counter.update(n => n + 1);
    const decrement = () => counter.update(n => n - 1);
</script>
```

```
<h3>value of count: { $counter}</h3>
<div>
    <button class="btn btn-increment" onclick={increment}>
        Increment
    </button>
    <button class="btn btn-decrement" onclick={decrement}>
        Decrement -
    </button>
</div>
```

Learning point

- Show the value of state containers from the store using the `$` prefix
  - `export const counter = …` ➔ `{ $counter }` – show value
  - `export const user = …` =➔ `{ $user.name }` – using objects
- Store objects are automatically subscribed to!
  - No manual subscription/unsubscription



Displaying the state

- Just import the state container from the store
- Use the `$`-prefix to show current value
- Value is updated automatically.
  - (see `StateReadOnlyComponent.svelte`)

```
<script lang="ts">
    // just importing the store
    import {counter} from "./store/counterStore";
</script>

<!--UI for this component-->
<h2>Reading from the state</h2>
value of count: { $counter}
```

Modern Svelte 5 Store

Using the top-level `$state()` rune to create store properties

Modern store – using Svelte 5 runes

- Create a top-level `xxx.svelte.ts` file using `$state(<T>)` rune
- The state holds an object, which properties are the store props.
- Classic stores are still around a lot (!) and useful for complex async streams or when you want more manual control over updates/subscriptions.

Similar example: `counterState.svelte.ts`

```
// counterstate.svelte.ts - note: no 'import's needed
export const counterState = $state({
    count: 0
});

// updates to the state
export const increment = () => { … }
export const decrement = () => { … }
export const reset = () => { … }
```

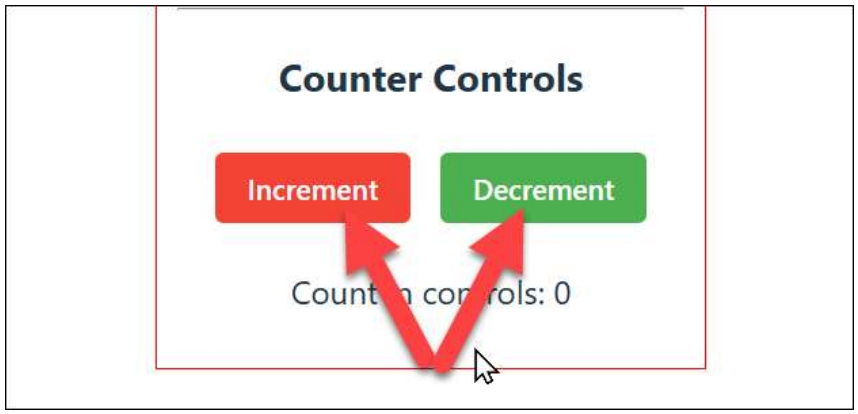
Note: inside the functions, the `counterState.count` should be updated!

Reading from the state

```
<!--CounterDisplay.svelte - showing state of counter-->
<script lang="ts">
    import {counterState} from "./store/counterstate.svelte";
</script>

<h3>Counter Display</h3>
<!--Note: no $ prefix needed-->
<p>Current count: {counterState.count}</p>
```

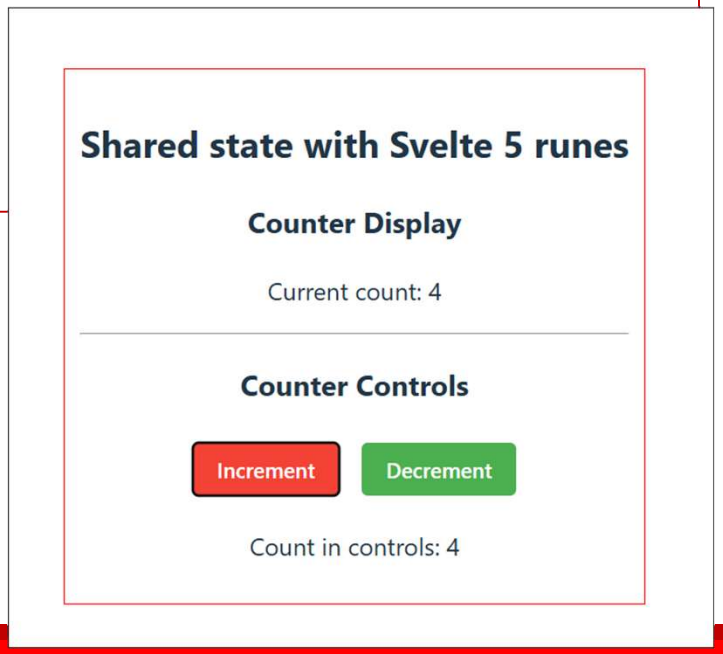
Updating the state

```
<script lang="ts">
    import {counterState, increment,
        decrement, reset} from './store/counterstate.svelte';
</script>

<h3>Counter Controls</h3>

<button class="btn btn-decrement"
    onclick={increment}>Increment</button>
<button class="btn btn-increment"
    onclick={decrement}>Decrement</button>
```



Putting it all together

```
<script lang="ts">
    import CounterDisplay from "./lib/CounterDisplay.svelte";
    import CounterControls from "./lib/CounterControls.svelte";
</script>

<!--HTML template-->
<div>
    <h2>Shared state with Svelte 5 runes</h2>
    <CounterDisplay/>
    <hr>
    <CounterControls/>
</div>
```



Summary

- There is one shared top-level object, annotated with `$state()`
- Components import the same state object
- Changing `counterState.count` updates the value in all components
- No `.subscribe()`, `.set()` or `.update()` necessary

- Classic store: state is a *wrapper object* around a value
- Runes store: state is the *value itself* – not a wrapper
  - There is no separate package or API for working with state. '*The value is just the value*', but shared among components

Official docs on `$state()`

SVELTE Docs Tutorial Packages Playground Blog search CTRL K

Introduction
Overview
Getting started
.svelte files
.svelte.js and .svelte.ts files

Runes
What are runes?
$state
$derived
$effect
$props
$bindable
$inspect
$host

Template syntax
Basic markup
{#if ...}
{#each ...}

SVELTE • RUNES

$state

SEE ALSO

Tutorial • Basic Svelte • Reactivity • State

The `$state` rune allows you to create *reactive state*, which means that your UI *reacts* when it changes.

```
<script>
  let count = $state(0);
</script>

<button onclick={() => count++}>
  clicks: {count}
</button>
```

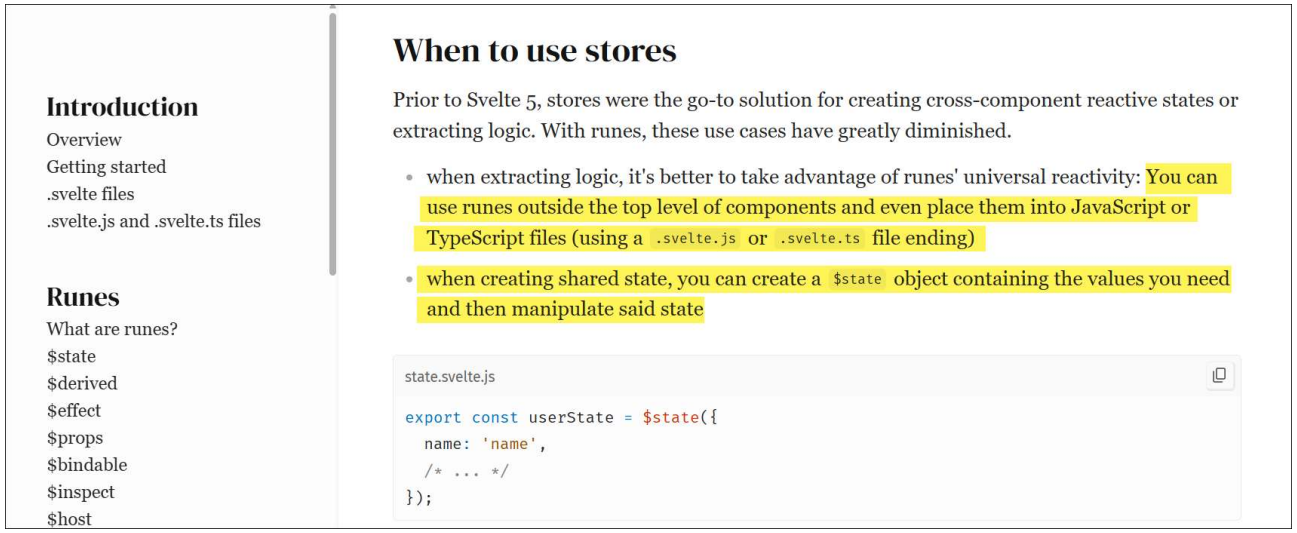
Unlike other frameworks you may have encountered, there is no API for interacting with state — `count` is just a number, rather than an object or a function, and you can update it like you

On this page
$state
$state.raw
$state.snapshot
$state.eager
Passing state into functions
Passing state across modules

https://svelte.dev/docs/svelte/%24state

Universal reactivity

"Runes bring universal reactivity. You can use them outside the top level of components and even in `.svelte.js` / `.svelte.ts` files. That is one of the main reasons Svelte 5 reduced the need for stores."



Introduction
Overview
Getting started
.svelte files
.svelte.js and .svelte.ts files

Runes
What are runes?
$state
$derived
$effect
$props
$bindable
$inspect
$host

When to use stores

Prior to Svelte 5, stores were the go-to solution for creating cross-component reactive states or extracting logic. With runes, these use cases have greatly diminished.

- when extracting logic, it's better to take advantage of runes' universal reactivity: You can use runes outside the top level of components and even place them into JavaScript or TypeScript files (using a `.svelte.js` or `.svelte.ts` file ending)
- when creating shared state, you can create a `$state` object containing the values you need and then manipulate said state

state.svelte.js

```
export const userState = $state({
  name: 'name',
  /* ... */
});
```

https://svelte.dev/docs/svelte/stores

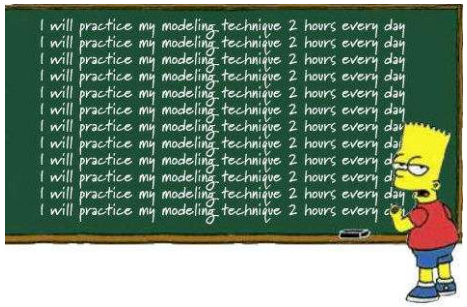
Summary – when to use?

- start with local `$state()`
- move to shared `$state()` module when needed
- use stores only when they add clear value
- "If value in one component, should be needed in a completely different component

"In Svelte 5, shared state often stops being "a store you talk to" and becomes "a reactive object you import and use."

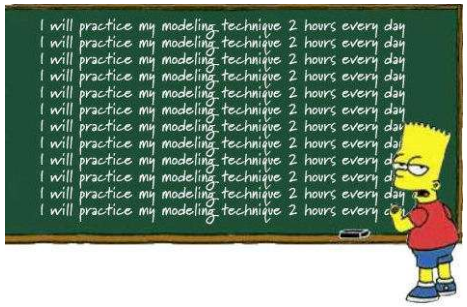
Workshop #1

- Work from: `../230-store-runes`, or use your own project
- Create a `userStore.ts` file
- Create a component where the user can type in a username
  - Store the username in the Store
- Create a component where the username is read from the store
- Updates in the first component, should be reflected in the second component

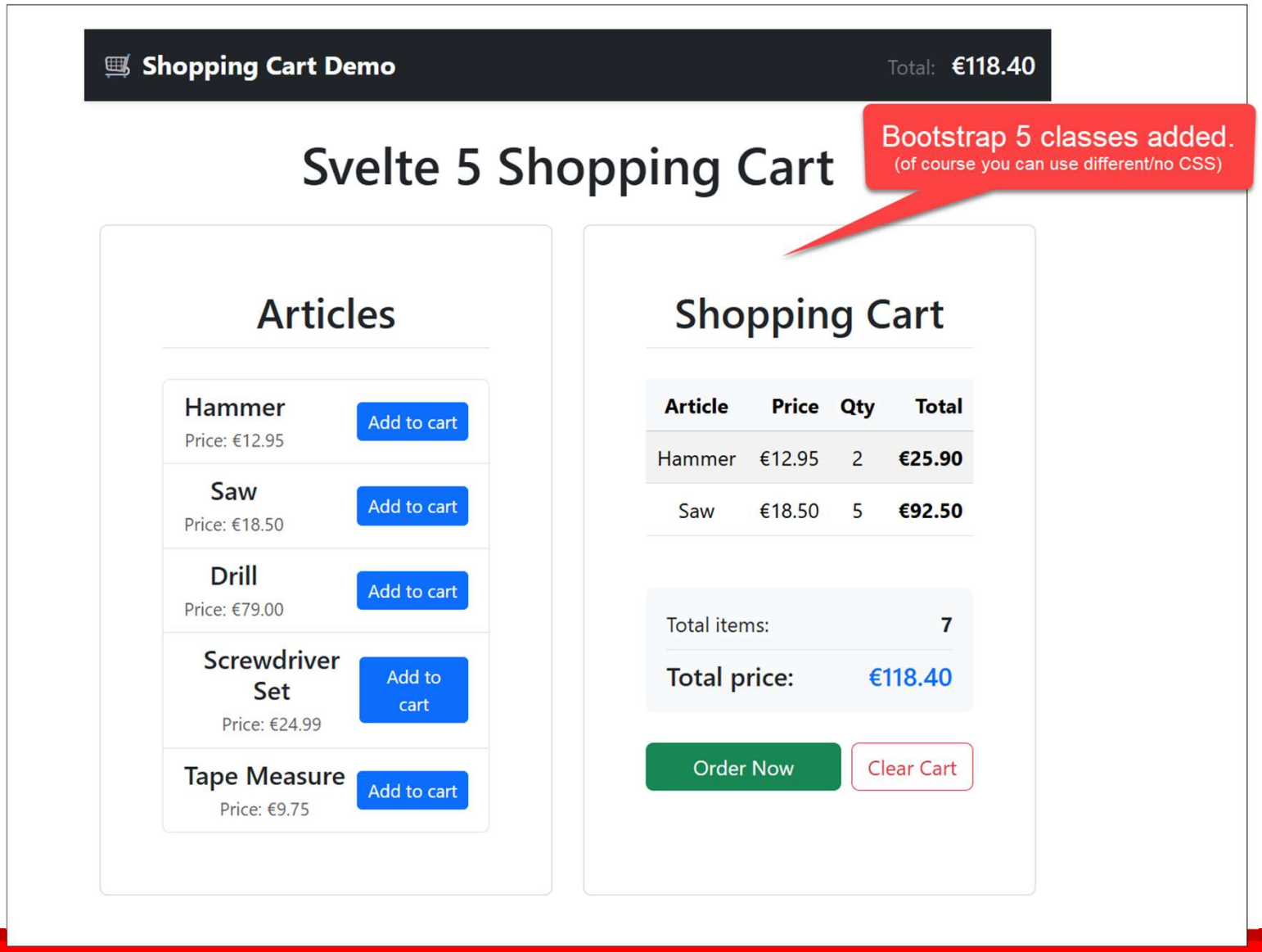


Workshop #2

- Create a new application with a store with objects and a ‘shopping cart’.
- The store should have an array with articles and a shopping cart.
- One component should show available articles and ‘Add to cart’ button
- One component should show the current content of the shopping cart
  - Show Total price of shopping cart
  - Buttons Place Order and Clear Cart
- One headercomponent should only show the shopping cart totals
  - (tip: use a `$derived.by()` for that)
- Working demo: `../320-store-shopping-cart`, but first try it yourself!



Example output



Checkpoint

- You know about different State/Store solutions in Svelte
  - Classic stores, using `.set()`, `.update()`
  - Runes stores, using `$state()`
- You can create stores in Svelte an (re)use them in components
- You can add, delete, update items in Stores and send them to a backend.